

Operator Overloading & Templates

Dry Run Practice — the WHY behind every line

This isn't a syntax cheat-sheet. Every dry run below walks through code line by line, shows you what the compiler actually does, and tells you exactly where students lose marks: wrong parameter count, missing `const`, missing `&`, dangling references, shallow copies, and self-assignment bugs.

- Part 1 — Operator Overloading: rules, member vs friend, references, `const`, dynamic memory, `mystring` quirks.
- Part 2 — Templates: type deduction, ambiguity, instantiation-time errors, reference parameters.
- Part 3 — Practice questions in the sessional style, with full answers.
- Linked lists are intentionally excluded — this file is 100% operator overloading + templates.

Reference material: your `mystring` / `newString` class examples and lecture slides. The sessional paper was used only to match the style of operator-overloading questions.

Part 0 — The Golden Rules (read this before every dry run)

Rule 1 — Who is the LEFT operand?

- If the left operand is an object of **your class** (e.g. `c1 + c2`, `c1 + 5`) → a **member function** works.
- If the left operand is **NOT** your class (e.g. `5 + c1`, or `cout << c1` where the left operand is an ostream) → it **cannot** be a member function. It must be a non-member (usually **friend**) function.
- Why: a member function is always called as `leftOperand.operator@(rightOperand)`. The compiler literally cannot call `5.operator+(c1)` — `int` has no member functions.

Rule 2 — How many explicit parameters?

- Member binary operator (e.g. `+`, `==`, `+=`): exactly **1** parameter. The left operand is the implicit `this`.
- Friend / non-member binary operator: exactly **2** parameters (neither operand is implicit).
- Member unary operator (e.g. prefix `++`, unary `-`): **0** parameters.
- Postfix `++/--` as a member: exactly **1** dummy `int` parameter that you never use — it exists only so the compiler can tell prefix and postfix apart.
- Getting the count wrong (0 when you need 1, or 2 when you need 1) is the single most common overloading mistake — it's a straight compile error, not a logic bug.

Rule 3 — Reference (&) or not?

- Parameters that are objects: pass as `const Type&` almost always. This avoids an extra copy-constructor call AND lets you pass literals / temporaries / const objects.
- Return type when you hand back `*this` (assignment, `+=`, `<<`, `>>`): return `Type&` (a reference) — needed for chaining like `a = b = c`.
- Return type when you hand back a brand-new local object (e.g. `operator+` building a sum): return `Type` **by value**, never by reference. The local variable dies when the function ends — a reference to it is a dangling reference.

Rule 4 — const correctness

- Mark a member function `const` if it does not modify the object: comparisons, accessors, `operator<<` helpers, the const overload of `operator[]`.
- Mark parameters `const Type&` so temporaries (like the literal 5 converted to a `Complex`) can bind to them — a plain `Type&` (non-const reference) refuses to bind to a temporary.

Rule 5 — Dynamic memory → Rule of Three

- The moment a class has a pointer member allocated with `new`, you must hand-write all three: **destructor**, **copy constructor**, **assignment operator**.
- Skip any one of these and the compiler's auto-generated version does a **shallow copy** (copies the pointer value, not the data) — this is the root cause of almost every mysterious crash / double-free / dangling-pointer bug in string classes.

Dry Run 1 — Member operator+ : c1+5 works, 5+c1 fails

PART 1 · MEMBER vs FRIEND

Code

```
class Complex {
    double re, im;
public:
    Complex(double r=0, double i=0): re(r), im(i) {}
    Complex operator+(const Complex& rhs) const {
        return Complex(re + rhs.re, im + rhs.im);
    }
};

int main() {
    Complex c1(2,3);
    Complex a = c1 + 5; // Line A
    Complex b = 5 + c1; // Line B
}
```

Trace

- Line A: compiler rewrites `c1 + 5` as `c1.operator+(5)`.
- ✓ `5` matches the parameter `const Complex&` because the single-argument constructor lets it convert `5` → `Complex(5,0)`. Compiles fine.
- Line B: compiler rewrites `5 + c1` as `5.operator+(c1)`.
- ✗ `int` has no member function `operator+`. There is also no matching non-member `operator+(int, Complex)`. Compile error: no match for `operator+`.

Key points

- The class type must be the LEFT operand for a member function to be chosen.
- A single-argument (non-explicit) constructor is what lets `c1+5` quietly work — `5` gets converted for you.
- This asymmetry (works one way, fails the other) is the classic giveaway that an operator was written as a member when it needed to be a friend.

✗ **Verdict: `5 + c1` fails to compile as written — fixed in Dry Run 2.**

Dry Run 2 — Fixing it with a friend function

PART 1 · THE FIX

Code

```

class Complex {
double re, im;
public:
Complex(double r=0, double i=0): re(r), im(i) {}
friend Complex operator+(const Complex& a, const Complex& b);
};
Complex operator+(const Complex& a, const Complex& b) {
return Complex(a.re + b.re, a.im + b.im); // friend can touch private data
}
int main() {
Complex c1(2,3);
Complex a = c1 + 5; // -> operator+(c1, Complex(5))
Complex b = 5 + c1; // -> operator+(Complex(5), c1)
}

```

Trace

- Now BOTH operands are explicit parameters — 2 parameters, as Rule 2 requires for a non-member binary operator.
- ✓ `c1 + 5` → `operator+(c1, 5)`; the int 5 converts to `Complex(5,0)` for the 2nd parameter.
- ✓ `5 + c1` → `operator+(5, c1)`; the int 5 converts to `Complex(5,0)` for the 1st parameter. Symmetric, both compile.

Key points

- friend gives a non-member function access to private members without adding public getters.
- Return type is `Complex` BY VALUE — a brand new object is built inside the function, so returning a reference would dangle (see Dry Run 12).
- Rule of thumb: if you want an operator to work symmetrically with mixed types, make it a friend/non-member.

✓ **Verdict: Both `c1+5` and `5+c1` now compile and give the same mathematical result.**

Dry Run 3 — `operator<<` and `operator>>` can never be members

PART 1 · STREAM OPERATORS

Code

```
// Attempt: make << a member function
class mystring {
char arr[100];
public:
mystring(const char* s) { strcpy(arr, s); }
ostream& operator<<(ostream& out) const { // WRONG placement
out << arr;
return out;
}
};
int main() {
mystring s1("abcd");
cout << s1; // does this even call our function?
}
```

Trace

- `cout << s1` is rewritten as `cout.operator<<(s1)` — the LEFT operand is `cout`, an `ostream`.
- ✗ We defined the member on `mystring`, not on `ostream`. `ostream` has no overload that accepts a `mystring`. Compile error.
- The correct version, matching your reference file: `friend ostream& operator<<(ostream& out, const mystring& s)` declared in the class, defined outside as a free function.

Key points

- Any operator whose LEFT operand is not your class type must be a non-member (friend if it needs private access).
- `operator<<` and `operator>>` are the textbook example: the left operand is always `ostream/istream`.
- Signature to memorize: `friend ostream& operator<<(ostream&, const T&);` and `friend istream& operator>>(istream&, T&);` — note `>>` takes `T` by non-const reference because it fills the object.

✗ **Verdict: Member << will not compile. It must be a friend/free function.**

Dry Run 4 — Parameter-count trap: single-parameter member operators

PART 1 · PARAMETER COUNT

Code

```

class mystring {
char arr[100];
public:
mystring(char s[]) { strcpy(arr, s); }
// ONE explicit parameter -> binary operator (this is the left operand)
void operator>>(int i) {
for (int k = 0; arr[k] != '\0'; k++) arr[k] += i;
}
};
int main() {
mystring s1("abcd");
s1 >> 1; // shifts every character by 1 -> "bcde"
}

```

Trace

- ✓ `s1 >> 1` → `s1.operator>>(1)`. Left operand = `s1` (implicit `this`), right operand = `1` (the single parameter). Two operands total — correct for a binary operator.
- If you had instead written `void operator>>(mystring, int)` as a MEMBER (2 explicit params), that's a compile error — a member binary operator only ever takes 1 explicit parameter.
- If you had written `void operator>>()` (0 params) as a member, the compiler treats it as a UNARY operator — `s1 >> 1` then fails because `>>` as written takes no arguments.

Key points

- “One parameter” is not a shortcut or a limitation — it's the correct count for a member binary operator. The mistake is forgetting the implicit `this` counts as operand #1.
- This particular overload (`>>` meaning “shift characters”) is a custom, non-standard use of `>>` — valid C++ (you can give operators any meaning you want), just not the usual stream-extraction meaning.
- Always ask: how many operands does this operator need in total, and is one of them going to be `this`?`

✓ **Verdict: One explicit parameter is exactly right for this member binary operator.**

Dry Run 5 — `const` in the parameter AND on the function — both matter

PART 1 · CONST CORRECTNESS

Code

```

class Complex {
double re, im;
public:
Complex(double r=0, double i=0): re(r), im(i) {}
bool operator==(Complex& rhs) { // missing const, twice
return re == rhs.re && im == rhs.im;
}
};
int main() {
const Complex c1(1,2);
const Complex c2(1,2);
if (c1 == c2) cout << "equal"; // Line X
}

```

Trace

→ Line X calls `c1.operator==(c2)`.

- ✗ `c1` is `const Complex`. Calling a NON-const member function on a const object is disallowed — the compiler assumes `operator==` might modify `c1`.
- ✗ Even if the function itself were `const`, the parameter `Complex& rhs` (non-const reference) cannot bind to `c2` which is also `const`. Two separate errors from two missing `const`s.

Key points

- Fixed signature: `bool operator==(const Complex& rhs) const` — `const` parameter so `const`/temporary objects can be passed in, `const` function so it can be called on `const` objects.
- Rule of thumb: any comparison, getter, or printing-helper operator should almost always end in `const`.
- This exact bug (forgetting `const`) is invisible until someone uses your class with a `const` object or a temporary — which is exactly what exam graders and test-cases do.

✗ **Verdict: Fails to compile twice over: non-const function on a const object, non-const& parameter refusing a const argument.**

Dry Run 6 — operator[] needs TWO overloads (const and non-const)

PART 1 · INDEXING

Code

```

class mystring {
    char* arr;
    int len;
public:
    mystring(const char* s) { len=strlen(s); arr=new char[len+1]; strcpy(arr,s); }
    char& operator[](int i) { return arr[i]; } // allows writing
    const char& operator[](int i) const { return arr[i]; } // read-only
    ~mystring() { delete[] arr; }
};
void printFirst(const mystring& s) {
    cout << s[0]; // Line P
}
int main() {
    mystring s1("abcd");
    s1[0] = 'Z'; // Line M
}

```

Trace

- Line M: `s1` is non-const, so overload resolution picks `char& operator[](int)`. It returns a real reference into the array, so assigning through it modifies `arr[0]`. Works.
- Line P: the parameter `s` is `const mystring&`. Overload resolution now picks `const char& operator[](int) const` — the only version callable on a const object.
- ✗ If you only wrote the non-const version, Line P fails to compile: “cannot call non-const member function on const object” — this is exactly the bug hiding in code that “works in `main()` but breaks when passed to a function that takes `const&`.”

Key points

- Whenever you write a non-const accessor operator that returns a reference into internal data, also write the const twin — otherwise your class silently breaks for any const parameter, which is very common (pass-by-const-reference is the default good practice!).
- The const version must return `const char&` (or a copy `char`), never a plain `char&` — that would let someone modify a supposedly-const object.

✓ **Verdict: Both overloads together make `[]` work in both mutable and read-only contexts.**

Dry Run 7 — Prefix vs postfix ++: reference return vs value return

PART 1 · INCREMENT/DECREMENT

Code

```

class Counter {
    int n;
public:
    Counter(int v=0): n(v) {}
    Counter& operator++() { // prefix: ++c
        ++n;
        return *this;
    }
    Counter operator++(int) { // postfix: c++ (dummy int)
        Counter temp = *this; // save the OLD value
        ++n;
        return temp; // must return by VALUE
    }
    int val() const { return n; }
};

```

Trace

- Prefix `++c` increments `n`, THEN gives you the object — there is no new object to build, so it correctly returns `Counter&` (reference to `*this`).
- Postfix `c++` must hand back the value BEFORE incrementing. That value is captured in a local variable `temp`.
- ✗ If postfix tried to `return temp;` as a reference (`Counter& operator++(int)`), `temp` is destroyed the instant the function returns — the caller receives a dangling reference. Undefined behavior.

Key points

- The dummy `int` parameter in postfix is never used — its only job is to give the postfix version a different signature from prefix so the compiler can tell `c++` from `++c`.
- General pattern: return a reference only when you're handing back something that already exists and will outlive the call (like `*this`). Return by value when you built something new/local.

✓ **Verdict: Prefix returns `Counter&`; postfix returns `Counter` by value — swapping these is a classic exam trap.**

Dry Run 8 — No user-defined copy constructor + dynamic memory = shallow copy crash

PART 1 · DYNAMIC MEMORY

Code

```
class mystring {
    char* arr;
public:
    mystring(const char* s) { arr = new char[strlen(s)+1]; strcpy(arr, s); }
    ~mystring() { delete[] arr; }
    // NO copy constructor written -> compiler generates one automatically
};
int main() {
    mystring s1("abcd");
    mystring s2 = s1; // Line C: which constructor runs?
} // Line D: both destructors run here
```

Trace

- Line C has no user-defined copy constructor, so the COMPILER-GENERATED one runs. It copies member-by-member: `s2.arr = s1.arr` — just the pointer value, not the characters.
- ✗ Now `s1.arr` and `s2.arr` point to the SAME heap block. Two objects, one buffer.
- ✗ Line D: `s2`'s destructor runs first (reverse order), does `delete[] arr` — the shared block is freed. Then `s1`'s destructor runs and does `delete[] arr` AGAIN on the same address — a double free. Undefined behavior / likely crash.

Key points

- This is exactly why the Rule of Three exists: a class with a raw pointer member needs a hand-written destructor, copy constructor, AND assignment operator, or none of the default ones are safe.
- The correct copy constructor allocates its OWN new buffer and copies the characters (deep copy), see the fixed `newString` reference: `strPtr = new char[strLength+1]; strcpy(strPtr, rightStr.strPtr);`

✗ **Verdict: Compiles and often even runs — then crashes (or silently corrupts memory) later. This is the hardest kind of bug to spot.**

Dry Run 9 — Removing & from a parameter — the mystring quirk you asked about

PART 1 · WHY & MATTERS

Code

```

class mystring {
    char* arr;
public:
    mystring(const char* s) { arr = new char[strlen(s)+1]; strcpy(arr, s); }
    ~mystring() { delete[] arr; }
    // Still no user-written copy constructor (shallow copy, as in Dry Run 8)
};

void printIt(mystring s) { // <-- passed BY VALUE, no &
    cout << "printing...\n";
} // s goes out of scope here -> its destructor runs

int main() {
    mystring s1("abcd");
    printIt(s1); // Line F
    cout << s1; // Line G: using s1 again
}

```

Trace

- Line F: `printIt(s1)` passes by value, so the (shallow, compiler-generated) copy constructor builds a temporary parameter `s` that shares `s1`'s pointer: `s.arr == s1.arr`.
- ✗ When `printIt` returns, `s` goes out of scope and its destructor runs `delete[] arr` — freeing the buffer that `s1.arr` ALSO points to.
- ✗ Line G: `s1.arr` is now a dangling pointer. `cout << s1` reads freed memory — garbage output, or a crash, depending on luck.

Key points

- This is precisely the bug behind “removing & causes an error”: with a raw pointer and no deep-copy constructor, EVERY pass-by-value silently creates a second owner of the same memory, and the first one to go out of scope frees it out from under everyone else.
- Fix #1 (preferred, and cheaper): change the parameter to `const mystring& s` — no copy is made at all, so there's nothing to free.
- Fix #2: write a correct deep-copy constructor (like Dry Run 8's fix). Then pass-by-value becomes SAFE again — just slower, because a real copy (new heap allocation) happens on every call.
- Note the difference from a FIXED-SIZE array `mystring` (`char arr[100]`, no pointer): there, the default copy just copies the whole array's bytes — that's a real deep copy already, so pass-by-value is safe (if wasteful) for that version. The danger is specific to pointer members.

✗ **Verdict: Without & and without a real copy constructor: undefined behavior after the call. Add & (or fix the copy constructor) to make it safe.**

Dry Run 10 — Assignment operator without a self-assignment check

PART 1 · OPERATOR=

Code

```

mystring& mystring::operator=(const mystring& rhs) {
    delete[] arr; // free our current buffer
    arr = new char[strlen(rhs.arr)+1]; // allocate new buffer
    strcpy(arr, rhs.arr); // copy from rhs
    return *this;
}
int main() {
    mystring s1("abcd");
    s1 = s1; // Line S: self-assignment
}

```

Trace

- Line S calls `s1.operator=(s1)` — here rhs IS s1 (same object, just referred to by a different name).
- ✗ `delete[] arr;` frees s1's buffer. But rhs.arr is the SAME pointer — it is now also deleted.
- ✗ `strcpy(arr, rhs.arr)` then reads from freed memory to build the new string. Garbage in, garbage out (or a crash).

Key points

- Fix: guard with `if (this != &rhs) { ...delete, allocate, copy... }` — compare addresses, not contents.
- Self-assignment looks pointless in `main()` but happens for real via aliases/pointers/references (e.g. `a[i] = a[i]` where `i==j`), so exams and test-suites always check for it.
- This is the `newString` reference implementation's exact pattern: `if (this != &rightStr) { delete [] strPtr; ... }`

✗ **Verdict: `s1 = s1` corrupts or crashes without the self-assignment guard.**

Dry Run 11 — `operator=` must return by reference, not by value or void

PART 1 · CHAINING

Code

```

mystring& mystring::operator=(const mystring& rhs) { // returns mystring&
    if (this != &rhs) {
        delete[] arr;
        arr = new char[strlen(rhs.arr)+1];
        strcpy(arr, rhs.arr);
    }
    return *this;
}
int main() {
    mystring s1("a"), s2("b"), s3("c");
    s1 = s2 = s3; // Line CH: right-to-left associativity
}

```

Trace

- `=` is right-associative: this becomes `s1 = (s2 = s3)`.

- ✓ `s2 = s3` runs first, and because `operator=` returns `mystring&` (a reference to `s2` itself), the expression `s2 = s3` evaluates to `s2`.
- ✓ Then `s1 = s2` runs using that result. Chaining works.
- ✗ If `operator=` returned `void` instead, `s1 = s2 = s3` would fail to compile — `s1=...` would have nothing to assign from.

Key points

- Always return `*this` by reference from `operator=`, `operator+=`, and similar — it's what makes chaining and further use of the expression's result possible.
- Returning by VALUE (a copy) would also compile and even chain correctly, but wastes a copy-construction on every link of the chain — reference is both correct-by-convention and more efficient.

✓ **Verdict: Reference return makes `s1 = s2 = s3` both compile and behave like assignment for built-in types.**

Dry Run 12 — `operator+` must return by VALUE — the opposite rule from `operator=`

PART 1 · DANGLING REFERENCE

Code

```
mystring& mystring::operator+(const mystring& rhs) { // WRONG: reference return
    mystring temp;
    strcpy(temp.arr, arr);
    strcat(temp.arr, rhs.arr);
    return temp; // temp is a LOCAL variable!
}
int main() {
    mystring s1("abcd"), s2("xyz");
    mystring s3 = s1 + s2; // Line R
    cout << s3; // undefined: reads a destroyed object
}
```

Trace

- Inside `operator+`, `temp` is a local (automatic/stack) object created fresh for this call.
- ✗ `return temp;` with return type `mystring&` hands back a reference to that local object.
- ✗ The instant the function returns, `temp`'s destructor runs and its stack memory is reclaimed. Line R now holds a reference to a destroyed object — using `s3` afterward is undefined behavior, even though it may “look fine” by accident.

Key points

- `operator+` (and anything that builds a brand-new result rather than modifying an existing object) must return **Type BY VALUE**: `mystring operator+(const mystring& rhs) { ...; return temp; }`
- This is the mirror image of Dry Run 11 — `operator=` returns a reference because `*this` already exists and outlives the call; `operator+` returns by value because `temp` does NOT outlive the call.

- Rule to memorize: returning `*this` (something that already existed before the call) → reference is safe. Returning a freshly-built local object → must be by value.

X Verdict: Compiles, but s3 ends up reading destroyed memory. Change the return type to `mystring` (no `&`) to fix it.

Dry Run 13 — Missing destructor — no crash, just a silent memory leak

PART 1 · DESTRUCTORS

Code

```
class mystring {
    char* arr;
public:
    mystring(const char* s) { arr = new char[strlen(s)+1]; strcpy(arr, s); }
    // no destructor written at all
};
void makeMany() {
    for (int i = 0; i < 100000; i++) {
        mystring temp("hello"); // Line L
    } // Line E: temp goes out of scope every iteration
}
```

Trace

- Line L allocates a new heap block on every single iteration.
- Line E: temp goes out of scope. Because there's no user-written destructor, the compiler-generated one runs — but it does nothing to arr, since arr is just a raw pointer (compiler doesn't know it “owns” heap memory).
- ✗ The heap block from Line L is never freed. After 100,000 iterations, 100,000 blocks are leaked — no crash, no error message, just growing memory usage until the program (or system) eventually struggles.

Key points

- A destructor is required the moment a class manages a resource (heap memory, a file handle, etc.) that the compiler doesn't know how to release on its own.
- Correct destructor: `~mystring() { delete[] arr; }` — matches the `new char[...]` array allocation with `delete[]`, not plain `delete`.
- Memory leaks are the quiet failure mode: the program “works” in every test you run by hand and only shows symptoms under sustained/looped use — exactly what graders' stress tests target.

✗ **Verdict: No compile error, no runtime crash in a short run — just a leak that grows without bound.**

Part 2 — Templates

Templates let one function/class definition work for many types. The catch: the compiler only checks the BODY of a template when it is actually used (instantiated) with a specific type — which is why template bugs often surface only for certain types, never for others.

Dry Run 14 — Type deduction: works, works, then silently fails

PART 2 · DEDUCTION

Code

```
template <class T>
T maxVal(T a, T b) {
    return (a > b) ? a : b;
}
int main() {
    cout << maxVal(3, 5); // Line 1
    cout << maxVal(3.5, 2.1); // Line 2
    cout << maxVal(3, 3.5); // Line 3
}
```

Trace

- ✓ Line 1: both arguments are int → the compiler deduces **T = int**. Instantiates maxVal(int,int). Fine.
- ✓ Line 2: both arguments are double → deduces **T = double**. Fine.
- ✗ Line 3: first argument is int, second is double. Template argument deduction needs ONE consistent T for both parameters — it will not silently convert 3 to 3.0 for you. Compile error: “no matching function, conflicting types for T”.

Key points

- Deduction happens per-parameter and then must agree across all parameters that share the same template letter.
- Fixes: call it explicitly as **maxVal<double>(3, 3.5)** (forces the conversion yourself), or declare two template parameters: **template<class T, class U> auto maxVal(T a, U b)** and decide the return type carefully.

✗ Verdict: Line 3 fails to compile — mixed types with a single template parameter is the #1 template gotcha.

Dry Run 15 — A template only fails for types that don't support the operator used inside

PART 2 · INSTANTIATION-TIME ERRORS

Code

```
template <class T>
T maxVal(T a, T b) {
    return (a > b) ? a : b; // needs operator> for T
}
struct Point { int x, y; }; // no operator> defined

int main() {
    cout << maxVal(3, 5); // Line A: fine, int has >
    Point p1{1,2}, p2{3,4};
    Point p3 = maxVal(p1, p2); // Line B
}
```

Trace

- ✓ Line A: `int` supports `>` natively — `T=int` instantiates cleanly.
- Line B: `T` is deduced as `Point`. The compiler now tries to actually generate the function body for `Point`, which requires `p1 > p2` to mean something.
- ✗ `Point` has no `operator>`. Error appears ONLY at the point of instantiation for `Point` — the template definition itself compiled without complaint, because the compiler doesn't check the body against every possible `T` up front.

Key points

- This is why the same template “works for numbers but breaks for my class” — it's not a bug in the template, it's a missing operator on the type you plugged in.
- Fix: give `Point` an `operator>` (member or friend, following all the Part 1 rules), or write a specialization / a comparator you pass in separately.
- Compiler error messages for this are notoriously long — read the FIRST error line (“no `operator>` matches these operands”); the rest is usually template-instantiation noise.

✗ **Verdict: Fails only when instantiated with a type that lacks the required operator — `int` is fine, `Point` is not.**

Dry Run 16 — Template parameters by value vs by reference

PART 2 · REFERENCE PARAMETERS

Code

```
template <class T>
void swapVals(T a, T b) { // BY VALUE - operates on local copies
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 1, y = 2;
    swapVals(x, y);
    cout << x << " " << y; // Line 0
}
```

Trace

- `swapVals(x, y)` makes local copies `a` and `b` of `x` and `y` (pass by value, just like a non-template function would).
- The swap happens between the local copies `a` and `b` inside the function.
- ✗ Function returns. Nothing was ever connected back to `x` and `y`. Line 0 still prints “1 2” — `x` and `y` are unchanged. No compile error, just a logic bug (looks like it works, does nothing).

Key points

- Fix: `template<class T> void swapVals(T& a, T& b)` — references bind directly to `x` and `y`, so modifying `a/b` modifies the caller's variables.
- This is the same by-value-vs-by-reference rule from Part 1, applied to templates: templates don't change the semantics of `&`, they just add type genericity on top.

X Verdict: Compiles and runs with no error — but silently does nothing useful. The classic “works except it doesn't” template bug.

Part 3 — Practice Questions (answers included)

Same style as your sessional-exam questions. Try to answer BEFORE reading the answer underneath.

Q1.

```
class Vector2D {
    double x, y;
public:
    Vector2D(double a=0, double b=0): x(a), y(b) {}
    Vector2D operator+(Vector2D rhs) {
        return Vector2D(x+rhs.x, y+rhs.y);
    }
};

int main() {
    Vector2D v1(1,2), v2(3,4);
    Vector2D v3 = v1 + v2; // (a)
    Vector2D v4 = v1 + 5; // (b) -- assume no int constructor overload matches
    implicitly here
    Vector2D v5 = 5 + v1; // (c)
}
```

Answer

- (a) Compiles — `v1.operator+(v2)`, both are `Vector2D`, member function applies directly.
- (b) Compiles IF the single-arg constructor allows `int`→`Vector2D` conversion (it does here, since only `x` is set: `Vector2D(5,0)`); otherwise a compile error (no viable conversion / no matching `operator+`).
- (c) Fails — left operand (`5`) is not a `Vector2D`, so the compiler looks for `int::operator+` or a matching free function `operator+(int, Vector2D)`; neither exists. Fix: make `operator+` a friend taking two `Vector2D` parameters.
- Style note: the parameter `Vector2D rhs` is by value here, not `const&` — it still compiles (just does one avoidable extra copy per call); prefer `const Vector2D& rhs`.

Q2.

```

class mystring {
char arr[50];
public:
mystring(const char s[]) { strcpy(arr, s); }
mystring operator+(mystring other) {
mystring temp;
strcpy(temp.arr, arr);
strcat(temp.arr, other.arr);
return temp;
}
void display() { cout << arr; }
};
int main() {
mystring s1("ab"), s2("cd");
mystring s3 = s1 + s2;
s3.display();
}

```

Answer

- Compiles and prints "abcd".
- `operator+` correctly returns by VALUE (Rule: new local object → value, never reference).
- Because `arr` is a fixed-size array (not a pointer), the default (compiler-generated) copy constructor used to build/return `temp` does a real byte-for-byte copy — no shallow-copy danger here, unlike the pointer-based `mystring` in Dry Runs 8–09.
- This class has no default constructor visible for `mystring temp`; to use — as written it would actually fail to compile (`temp` needs a no-argument constructor, and only a single-arg one is defined). A correct answer adds `mystring() { arr[0] = '\0'; }`.

Q3.

```

class Fraction {
int num, den;
public:
Fraction(int n, int d): num(n), den(d) {}
bool operator<(Fraction other) {
return (double)num/den < (double)other.num/other.den;
}
};
int main() {
const Fraction f1(1,2), f2(3,4);
if (f1 < f2) cout << "f1 smaller";
}

```

Answer

- Fails to compile: `f1` and `f2` are `const Fraction` objects, but `operator<` is not marked `const`, so it cannot be called on either of them.
- Also, the parameter `Fraction other` should be `const Fraction& other` — not strictly required to compile here (no `const`/temporary is passed as the argument), but it's the correct, efficient form and is exactly the kind of thing exam rubrics dock marks for.
- Fix: `bool operator<(const Fraction& other) const { ... }`

Q4.

```
template <class T>
T sumArray(T arr[], int n) {
    T total = 0;
    for (int i = 0; i < n; i++) total = total + arr[i];
    return total;
}

int main() {
    int a[] = {1,2,3};
    cout << sumArray(a, 3) << endl; // (a)

    string b[] = {"a","b","c"};
    cout << sumArray(b, 3) << endl; // (b)
}
```

Answer

- (a) T deduced as int. `T total = 0;` works (0 converts to int fine). Output: 6.
- (b) T deduced as string. `T total = 0;` tries `string total = 0;` — string has no constructor taking a plain int literal 0 in this way (this specific initialization is NOT valid for `std::string`), so this line fails to compile only when instantiated with string.
- This is the same lesson as Dry Run 15: the template body compiles fine in the abstract, but instantiating it for a type that doesn't support an operation used inside (here, constructing from 0) fails at instantiation time, not at template-definition time.

Q5.

```
class Box {
    int* data;
public:
    Box(int v) { data = new int(v); }
    ~Box() { delete data; }
    Box(const Box& other) { data = other.data; } // (a)
    Box& operator=(const Box& other) { // (b)
        data = other.data;
        return *this;
    }
};
```

Answer

- (a) The copy constructor copies the POINTER, not the int it points to — shallow copy. Two Box objects now share one heap int; when both are destroyed, the second delete is a double-free.
- (b) The same shallow-copy problem, PLUS it never frees the object's own old `data` before overwriting the pointer — that old int is leaked, and it also has no self-assignment guard.
- Correct copy constructor: `data = new int(*other.data);`
- Correct operator=: `if (this != &other) { delete data; data = new int(*other.data); } return *this;`

Q6.

```

class Matrix {
int rows, cols;
public:
Matrix(int r, int c): rows(r), cols(c) {}
int& operator()(int r, int c) { /* ... */ }
};
int main() {
Matrix m(3,3);
m(1,1) = 5; // (a)
cout << m(1,1); // (b)
}

```

Answer

- (a) Compiles: `operator()` is a member with 2 explicit parameters — valid, since function-call operator `()` is one of the few operators allowed any number of parameters (it isn't limited to the usual binary-operator "1 parameter" rule).
- (b) Also compiles if `m` is non-const — same overload is reused for both reading and writing since it returns `int&`.
- If `m` were declared `const Matrix m(3,3);`, line (b) would fail without an additional `int operator()(int,int) const` overload — exactly the same const-overload pattern as `operator[]` in Dry Run 6.

End of practice set. Re-do Dry Runs 8–12 from memory (dynamic memory + & + return-by-value/reference) — that cluster is where most marks are lost.